

ФГБОУ ВО «Пензенский государственный университет»

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по дисциплине «Логика и ОА»

лабораторная работа №4 на тему:

«Бинарное дерево поиска»

Выполнил студенты группы 25ВВВ3:

Коновалов А. С

Шорохов В. В

Приняли:

к.т.н., доцент Юрова О. В.

к.т.н., доцент Деев М. В.

Тема: «Бинарное дерево поиска»

Цель работы: изучить динамическую структуру данных «бинарное дерево поиска» на языке C, освоить работу с рекурсивными функциями и указателями. Реализовать построение дерева, вывод на экран, поиск значения, подсчёт числа вхождений, добавление без дубликатов. Оценить вычислительную сложность процедуры поиска.

Лабораторное задание:

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Пояснительный текст:

Задание 1. В структуру Node входят три поля: `int data` — полезная информация (целое число), `struct Node *left` и `struct Node *right` — указатели на левого и правого потомка. Корень дерева хранится в глобальном указателе `struct Node *root`.

Дерево в методичке строится по правилу: значение левого потомка больше значения родителя, значение правого потомка меньше значения родителя. Поэтому и поиск идёт по тому же правилу — влево, если искомое больше, и вправо, если меньше.

Ключевая функция `find_node()` работает рекурсивно:

- если текущий узел `NULL` — значит, дошли до конца ветки, элемент не найден, возвращаем `NULL`;
- если `r->data == value` — нашли, возвращаем указатель на узел;
- если `value > r->data` — уходим в левое поддерево;
- если `value < r->data` — уходим в правое поддерево.

Остальные функции (`CreateTree`, `print_tree`) совпадают с базовым примером из методички.

Сложность: $O(h)$, где h — высота дерева. В среднем для сбалансированного дерева $O(\log n)$, в худшем (вырожденное дерево) — $O(n)$.

Результат работы программы:

```
Введите значение для поиска: 7
Элемент 7 найден
```

Задание 2:

Функция `count_occurrences()` обходит всё дерево целиком и считает количество узлов, у которых `data` совпадает с заданным значением:

- если узел `NULL` — возвращаем 0;
- если `r->data == value` — в текущем узле совпадение, `count = 1`;
- результат = `count` + число совпадений в левом поддереве + число совпадений в правом поддереве.

Обход идёт по обеим веткам, потому что дубликаты в дереве могут оказаться и слева, и справа — в отличие от классического BST, дерево из методички не гарантирует, что дубликат окажется строго в одной ветке.

Сложность: $O(n)$ — обходим все узлы дерева.

Результат работы программы:

```
Введите значение для подсчёта вхождений: 5
Элемент 5 встречается 1 раз
```

Задание 3:

Функция `CreateTreeUnique()` отличается от базовой `CreateTree()` одной проверкой, добавленной перед рекурсивным вызовом:

Если значение уже присутствует в дереве — функция просто возвращает `root` без добавления нового узла, а пользователю выводится сообщение о пропуске. Это позволяет избежать появления дубликатов.

Чтобы можно было сравнить поведение обеих функций, в главном меню добавлен переключатель режима «без дубликатов». В режиме с дубликатами вызывается `CreateTree()`, в режиме без — `CreateTreeUnique()`.

Сложность: $O(h)$ — такая же, как у обычного добавления, поскольку проверка происходит по ходу спуска.

Результат работы программы:

```
Режим: без дубликатов
Введите число: 1
Элемент 1 уже есть в дереве, пропущен
```

Задание 4:

Сложность поиска в бинарном дереве поиска зависит от его высоты h :

- Средний случай (сбалансированное дерево): $h \approx \log_2 n \rightarrow O(\log n)$.
- Худший случай (вырожденное дерево, когда каждый новый элемент больше или меньше предыдущего): $h = n \rightarrow O(n)$.

Каждый шаг поиска отсекает половину дерева — но только если оно сбалансировано. Если дерево вырожденное (например, добавляли числа по возрастанию: 1, 2, 3, 4, 5), оно превращается в длинную цепочку, и поиск идёт по одному элементу за шаг — как в обычном списке.

Для демонстрации реализована функция `find_with_steps()`, которая работает так же, как `find_node()`, но дополнительно считает количество шагов. В меню пункт «Оценка сложности поиска» выводит число шагов для конкретного значения и напоминает теоретические оценки.

Результат работы программы:

```
Введите значение для поиска: 7
Элемент 7 найден за 7 шагов

Теоретическая оценка:
- Сбалансированное дерево:  $O(\log n)$ 
- Вырожденное дерево:       $O(n)$ 
- Ваш результат:           7 шагов
```

Вывод:

В ходе лабораторной работы была изучена динамическая структура данных «бинарное дерево поиска» на языке C и освоены рекурсивные функции для работы с ней. На основе базового кода из методички реализованы:

1. Поиск значения — рекурсивный спуск от корня с отсечением половины дерева на каждом шаге. Сложность $O(\log n)$ в среднем.
2. Подсчёт вхождений — полный обход дерева с подсчётом совпадений. Сложность $O(n)$.
3. Добавление без дубликатов — в функцию добавления введена проверка на равенство значений, что позволяет исключить повторные вхождения.
4. Оценка сложности поиска — на практике подтверждено, что количество шагов зависит от высоты дерева: $O(\log n)$ для сбалансированного и $O(n)$ для вырожденного.

Экспериментально подтверждено, что бинарное дерево поиска позволяет эффективно хранить данные в отсортированном виде и обеспечивать быстрый доступ к элементам при условии сбалансированности. Полученные навыки работы с рекурсивными структурами являются базовыми для построения более сложных структур данных: AVL-деревьев, красно-чёрных деревьев, B-деревьев.